UPF EDITORA
UNIVERSIDADE DE PASSO FUNDO

RBCA
ISSN 2176-6649

Revista Brasileira de Computação Aplicada, August, 2026

DOI: 10.5335/rbca.vxxix.9999
Vol. xx, Nº x, pp. 1–9
Homepage: seer.upf.br/index.php/rbca/index

ORIGINAL PAPER

# Alocação Dinâmica de Memória no FreeRTOS: Comparativo entre First-Fit e Best-Fit no ESP32

# Dynamic Memory Allocation in FreeRTOS: A Comparison of First-Fit and Best-Fit on the ESP32

Paulo Sérgio Furtado Filho [1] and Paulo Sérgio Furtado [2]

[1]Instituto Federal de Santa Catarina, [2]Universidade do Estado de Santa Catarina

*paulo.sff@aluno.ifsc.edu.br; paulo.furtado@udesc.br

**Recebido:** yyyy-mm-dd. **Revisado:** yyyy-mm-dd. **Aceito:** yyyy-mm-dd.

## Resumo

Sistemas embarcados e dispositivos de Internet das Coisas (IoT) operam sob restrições de memória e processamento, exigindo previsibilidade temporal em sistemas operacionais de tempo real (RTOS). Este trabalho apresenta uma avaliação experimental comparativa das estratégias de alocação dinâmica *First-Fit* (heap_4) e *Best-Fit* (heap_2) do FreeRTOS, executadas no microcontrolador dual-core ESP32. Foi desenvolvido um ambiente de testes controlado e estocástico utilizando ESP-IDF v5.3.1, simulando requisições de memória com diferentes tamanhos e liberações não ordenadas. Foram analisadas 58.068 operações de memória, coletadas em 30 baterias independentes de testes distribuídas em duas placas físicas. Avaliaram-se os tempos médios de alocação e desalocação, o desvio padrão (*jitter*), o pior caso (*Worst-Case Execution Time* — WCET) e o percentil 99 (P99). Os resultados demonstram que o *Best-Fit* apresentou melhor desempenho na alocação, com média de 6,95 µs e WCET de 21 µs, enquanto o *First-Fit* obteve melhor desempenho na desalocação, com média de 5,63 µs e WCET de 16 µs, além de desvio padrão de 0,72 µs. Conclui-se que o *First-Fit* apresenta vantagens para aplicações de tempo real de longa duração, enquanto o *Best-Fit* mostra-se adequado para fases de inicialização.

**Palavras-Chave**: FreeRTOS; Alocação Dinâmica de Memória; ESP32; First-Fit; Best-Fit

## Abstract

Embedded systems and Internet of Things (IoT) devices operate under memory and processing constraints, requiring temporal predictability in real-time operating systems (RTOS). This work presents a comparative experimental evaluation of the First-Fit (heap_4) and Best-Fit (heap_2) dynamic memory allocation strategies of FreeRTOS, running on the dual-core ESP32 microcontroller. A controlled and stochastic testing environment was developed using ESP-IDF v5.3.1, simulating memory requests with different sizes and out-of-order deallocations. A total of 58,068 memory operations were analyzed, collected from 30 independent test runs distributed across two physical boards. Average allocation and deallocation times, standard deviation (jitter), Worst-Case Execution Time (WCET), and the 99th percentile (P99) were evaluated. The results demonstrate that Best-Fit achieved better allocation performance, with an average of 6.95 µs and a WCET of 21 µs, while First-Fit achieved better deallocation performance, with an average of 5,63 µs and a WCET of 16 µs, as well as a standard deviation of 0.72 µs. It is concluded that First-Fit offers advantages for long-running real-time applications, while Best-Fit is suitable for initialization phases.

**Keywords**: FreeRTOS; Dynamic Memory Allocation; ESP32; First-Fit; Best-Fit

## 1 Introdução

Os sistemas embarcados ocupam hoje um espaço central na computação, presentes em dispositivos que vão de sensores industriais a wearables de saúde, sustentando a expansão da Internet das Coisas (IoT). Esses dispositivos costumam operar sob restrições severas de memória, energia e capacidade de processamento, características que os diferenciam radicalmente dos computadores de propósito geral Musaddiq et al. (2018). Para lidar com essas limitações, a indústria e a academia desenvolveram sistemas operacionais de tempo real (RTOS) enxutos, capazes de garantir previsibilidade temporal mesmo em hardware modesto. Entre eles, o FreeRTOS se consolidou como uma das soluções mais adotadas em microcontroladores de baixo custo, como o ESP32, por oferecer um núcleo compacto, escalonamento preemptivo baseado em prioridades e baixo consumo de recursos Goyette (2007).

Um dos aspectos mais sensíveis no projeto desses sistemas é o gerenciamento de memória dinâmica. Diferentemente de sistemas de propósito geral, em que o alocador prioriza o desempenho médio, sistemas embarcados de tempo real exigem que as operações de alocação e desalocação possuam tempo de execução limitado e previsível (WCET), já que atrasos inesperados podem comprometer prazos críticos da aplicação Puaut (2002). Historicamente, isso levou muitos projetistas a evitar completamente a alocação dinâmica, optando por alocação estática definida em tempo de compilação. Entretanto, aplicações modernas de IoT, que envolvem streaming de dados, comunicação em rede e até aprendizado de máquina embarcado, tornam essa abordagem cada vez menos viável, pressionando os desenvolvedores a recorrer novamente a alocadores dinâmicos, mesmo em ambientes com poucos kilobytes de RAM Ramakrishna et al. (2008); Tanović e Mezei (2024).

Entre as estratégias clássicas de alocação dinâmica, First-Fit e Best-Fit permanecem como referência tanto para uso prático quanto para comparação com técnicas mais recentes. O First-Fit percorre a lista de blocos livres e aloca no primeiro espaço suficientemente grande, enquanto o Best-Fit busca o menor bloco livre capaz de atender à requisição, minimizando o resto não utilizado Shore (1975). Embora sejam algoritmos simples e de baixo overhead computacional, seu comportamento em termos de fragmentação de memória e tempo de resposta não é trivial: comparações publicadas ao longo de décadas mostram resultados nem sempre consistentes entre si, variando conforme a distribuição estatística das requisições e as características da carga de trabalho Shore (1975).

Esse cenário se torna ainda mais relevante quando se observa a evolução recente dos alocadores para sistemas de tempo real. Propostas como o TLSF (Two-Level Segregated Fit) e, mais recentemente, o RT-Mimalloc buscam oferecer tempos de alocação limitados a custo de maior complexidade estrutural, adaptando técnicas de alocadores de propósito geral ao contexto de tempo real Crespo et al. (2006); Giannessi et al. (2024). Ainda assim, estratégias clássicas como First-Fit e Best-Fit continuam sendo o mecanismo padrão de diversos RTOS de mercado, incluindo o alocador nativo do FreeRTOS, o que reforça a necessidade de compreender seu comportamento em plataformas amplamente utilizadas, como o ESP32.

Apesar da relevância do tema, ainda existem lacunas importantes na literatura quanto ao comportamento específico dessas estratégias em microcontroladores de recursos extremamente limitados. Segundo Puaut (2002), a maior parte dos estudos sobre desempenho de alocadores dinâmicos concentra-se na avaliação do caso médio, deixando em segundo plano o comportamento de pior caso, que é justamente o mais relevante para sistemas de tempo real crítico. Essa lacuna é agravada em ambientes sem unidade de gerenciamento de memória (MMU), como é o caso da maioria dos microcontroladores, nos quais a fragmentação de memória não pode ser corrigida por meio de compactação ou realocação de páginas Deligiannis e Kornaros (2016).

Outro obstáculo apontado na literatura é a escassez de estudos que avaliem, de forma controlada e comparativa, o comportamento de First-Fit e Best-Fit especificamente em hardware real de baixo custo, em contraposição a simulações genéricas ou avaliações em arquiteturas de desktop. Bukkapatnam et al. (2020) destacam que, em sistemas embarcados sem MMU, a escolha do algoritmo de alocação impacta diretamente tanto a velocidade de resposta quanto o nível de fragmentação residual, mas poucos trabalhos quantificam esse impacto sob condições de carga realistas em plataformas amplamente utilizadas comercialmente. Já Ramakrishna et al. (2008) argumentam que os algoritmos básicos de alocação, embora bem descritos teoricamente, carecem de avaliações experimentais que considerem simultaneamente o tempo de resposta e o uso de memória sob cargas de trabalho representativas de aplicações reais.

Por fim, mesmo estudos recentes conduzidos diretamente sobre o ESP32 tendem a tratar a alocação de memória como uma variável secundária, voltando o foco para outros aspectos do sistema, como desempenho de algoritmos de aprendizado de máquina ou análise de jitter em arquiteturas multi-core, sem aprofundar a comparação entre estratégias de alocação propriamente ditas Arm et al. (2022); Tanović e Mezei (2024). Esse conjunto de lacunas evidencia um obstáculo concreto para desenvolvedores de sistemas embarcados de tempo real: a escolha entre First-Fit e Best-Fit no FreeRTOS ainda carece de evidências experimentais claras, obtidas diretamente em microcontroladores de recursos limitados, que orientem essa decisão de projeto considerando previsibilidade, fragmentação e desempenho sob restrições reais de hardware.

Diante desse cenário, justifica-se a realização deste trabalho, que se propõe a preencher a lacuna identificada por meio de uma avaliação experimental controlada do First-Fit e do Best-Fit diretamente em um microcontrolador de recursos limitados, o ESP32, executando o FreeRTOS. Ao contrário das análises predominantemente teóricas ou baseadas em simulação encontradas na literatura Shore (1975); Puaut (2002), a proposta aqui apresentada mede o comportamento real dos dois alocadores sob cargas de trabalho controladas, considerando não apenas o desempenho médio, mas também o pior caso e a previsibilidade, aspectos historicamente negligenciados apesar de serem determinantes para aplicações de tempo real crítico. Espera-se, com isso, gerar evidências práticas que auxiliem desenvolvedores de sistemas embarcados na escolha da es-

tratégia de alocação de memória mais adequada ao projetar aplicações de IoT e tempo real em plataformas de baixo custo, contribuindo tanto para a literatura acadêmica sobre gerenciamento de memória em ambientes sem MMU quanto para decisões de engenharia em cenários reais de desenvolvimento.

Diante da lacuna identificada, este trabalho tem como objetivo avaliar e comparar o desempenho das estratégias de alocação de memória First-Fit e Best-Fit em sistemas embarcados de tempo real, utilizando o microcontrolador ESP32 e o sistema operacional FreeRTOS como ambiente experimental, a fim de fornecer evidências práticas que auxiliem na escolha da estratégia de alocação mais adequada para aplicações de tempo real com recursos limitados. Para alcançar esse objetivo, o trabalho parte de uma revisão da literatura sobre estratégias de alocação de memória dinâmica, com foco em First-Fit, Best-Fit e algoritmos correlatos, buscando compreender o funcionamento do gerenciador de memória do FreeRTOS no ESP32 e identificar como e onde essas estratégias podem ser configuradas ou substituídas. A partir desse embasamento, são elencadas as métricas de desempenho relevantes para a comparação, tempo de alocação, fragmentação de memória, uso de heap e previsibilidade/jitter, as quais orientam a implementação de um ambiente experimental no ESP32 capaz de executar cargas de trabalho com padrões controlados de alocação e desalocação de memória. Por fim, o desempenho das duas estratégias é medido sob as mesmas condições de teste e comparado entre si, discutindo-se vantagens, limitações e adequação de cada uma para aplicações de tempo real em ambientes com recursos limitados.

## 2 Revisão bibliográfica:

Esta seção reúne e analisa as contribuições da literatura relevantes ao problema de pesquisa, organizadas em quatro eixos temáticos. A Tabela 1 sintetiza os principais estudos discutidos e posiciona a contribuição deste trabalho: fundamentos da alocação dinâmica de memória e o comportamento de First-Fit e Best-Fit; alocadores voltados a sistemas de tempo real e embarcados sem unidade de gerenciamento de memória (MMU); o gerenciamento de memória no FreeRTOS e no ESP32 especificamente; e avaliações experimentais comparativas entre estratégias clássicas de alocação. Esses eixos oferecem o embasamento teórico necessário para justificar as escolhas metodológicas adotadas neste trabalho.

### 2.1 Fundamentos da alocação dinâmica de memória

O estudo comparativo entre First-Fit e Best-Fit remonta ao trabalho seminal de Shore (1975), que, por meio de simulação, investigou a fragmentação externa produzida pelas duas estratégias.

O autor demonstrou que a eficiência de tempo-memória de ambas costuma ficar dentro de 1 a 3% uma da outra, mas que o desempenho relativo depende fortemente da distribuição estatística das requisições: para distribuições exponenciais e hiperexponenciais, o First-Fit tende a superar o Best-Fit, enquanto para distribuições normais e uniformes ocorre o inverso.

Esse resultado é relevante para o presente trabalho porque evidencia que comparações entre as duas estratégias não podem ser generalizadas sem considerar o padrão de carga utilizado, o que reforça a necessidade de definir cuidadosamente o workload experimental empregado nos testes com o ESP32.

Décadas mais tarde, Puaut (2002) reformulou o problema sob a ótica de sistemas de tempo real, argumentando que a maior parte dos estudos sobre alocadores dinâmicos concentra-se no caso médio, negligenciando o comportamento de pior caso (WCET), que é o que efetivamente importa para a schedulability analysis de aplicações críticas.

Ao comparar analiticamente o pior caso teórico com medições reais obtidas pela execução de cargas sintéticas e reais, a autora forneceu diretrizes para orientar desenvolvedores na escolha entre usar ou não alocação dinâmica, e qual algoritmo preferir do ponto de vista da previsibilidade.

Essa lacuna, a escassez de estudos focados em pior caso e previsibilidade, e não apenas em desempenho médio, é um dos pilares que justificam a metodologia deste trabalho, que inclui métricas de jitter e pior caso além da média.

### 2.2 Alocadores para sistemas de tempo real e embarcados sem MMU

A partir dos fundamentos clássicos, uma parte significativa da literatura voltou-se ao desenvolvimento de alocadores especializados para ambientes de tempo real.

Crespo et al. (2006) propuseram o TLSF (Two-Level Segregated Fit), um algoritmo projetado especificamente para RTOS, capaz de garantir tempo de alocação limitado, O(1), mantendo baixa fragmentação, em contraste com os alocadores de propósito geral que priorizam desempenho médio em detrimento de previsibilidade.

Na mesma linha, Giannessi et al. (2024) revisitaram o problema quase vinte anos depois, adaptando o alocador de propósito geral Mimalloc para criar o RT-Mimalloc, que busca aliar tempos de alocação mais previsíveis ao desempenho médio competitivo dos alocadores modernos, os autores destacam que poucos esforços recentes foram dedicados a alocadores de tempo real com garantias formais, o que motiva a continuidade de pesquisas nessa área mesmo décadas após o TLSF.

Voltando-se especificamente a ambientes sem MMU, cenário direto do ESP32, Deligiannis e Kornaros (2016) propuseram o Adaptive Memory Management Scheme (AMM), alcançando speedups de 2x a 5x sobre o alocador padrão da GLIBC na alocação de blocos, ao custo de uma desalocação 20% mais lenta, mas com fragmentação praticamente eliminada.

Bukkapatnam et al. (2020) seguiram direção semelhante com o Smart Memory Management (SaMM), voltado a sistemas embarcados sem MMU, reforçando que, nesse tipo de ambiente, a fragmentação não pode ser corrigida por compactação de páginas, tornando a escolha do algoritmo de alocação uma decisão de projeto ainda mais crítica.

**Tabela 1:** Síntese dos trabalhos relacionados e posicionamento deste estudo.

| Estudo | Ambiente / Plataforma | Foco Principal | Hardware Real + RTOS sem MMU? |
|---|---|---|---|
| Shore (1975) | Simulação Teórica | Fragmentação externa de First-Fit e Best-Fit | Não |
| Puaut (2002) | Cargas Sintéticas/Reais | WCET e previsibilidade teórica em RT | Não |
| Crespo et al. (2006) | RTOS Genérico | Algoritmo TLSF de tempo limitado $O(1)$ | Não |
| Ramakrishna et al. (2008) | Hardware Embarcado | Alocador inteligente vs. clássicos (MIS) | Não |
| Deligiannis e Kornaros (2016) | SoC sem MMU | Esquema Adaptativo (AMM) vs. GLIBC | Não |
| Bukkapatnam et al. (2020) | Sistema sem MMU | Smart Memory Management (SaMM) | Não |
| Arm et al. (2022) | ESP32 Dual-Core | Jitter de tarefas e troca de contexto | Não (Memória secundária) |
| Tanović e Mezei (2024) | ESP32 / FreeRTOS | Paralelização de $k$-means / PSRAM e Flash | Não (Memória secundária) |
| Septian et al. (2026) | Simulador Mishal23 | Eficiência de First-Fit, Best-Fit e Worst-Fit | Não |
| **Este Trabalho (2026)** | **ESP32 Real / FreeRTOS** | **First-Fit vs. Best-Fit (Média, Jitter, WCET e P99)** | **SIM** |

*Avalia experimentalmente First-Fit vs. Best-Fit com medições de pior caso e jitter diretamente sobre hardware microcontrolado comercial executando um kernel RTOS em tempo de execução real.
Fonte: Os autores (2026).

Já Rebel et al. (2013) atacaram o problema em um extremo mais restrito de recursos, microcontroladores com poucos kilobytes de RAM, propondo um alocador de tempo constante baseado em pools de memória e cabeçalhos, mostrando que soluções especializadas podem superar os alocadores dinâmicos convencionais (CDMA) mesmo em hardware ultra limitado.

Em conjunto, esses trabalhos evidenciam uma tendência clara: quanto mais restrito o ambiente, maior a motivação para abandonar alocadores genéricos em favor de soluções sob medida, o que reforça a relevância de investigar diretamente as opções nativas do FreeRTOS, em vez de assumir seu comportamento a partir de alocadores de propósito geral.

## 2.3 Gerenciamento de memória no FreeRTOS e no ESP32

No nível do sistema operacional, Musaddiq et al. (2018) realizaram uma revisão abrangente sobre gerenciamento de recursos em sistemas operacionais para IoT, comparando Contiki, TinyOS e FreeRTOS quanto a suas estratégias de gerenciamento de processos, memória, energia e comunicação.

Os autores destacam o FreeRTOS como uma das soluções mais adotadas para dispositivos com recursos escassos, mas apontam que a documentação sobre o comportamento interno de seus mecanismos de alocação de memória permanece limitada, lacuna também identificada por Goyette (2007), cuja análise funcional detalhada do núcleo do FreeRTOS revelou que o sistema oferece esquemas básicos de alocação de memória, mas deixa a critério do desenvolvedor a escolha entre eles, sem orientação experimental clara sobre qual estratégia é mais adequada a cada cenário.

Estudos mais recentes conduzidos diretamente sobre o ESP32 tendem a tratar a alocação de memória como variável secundária. Arm et al. (2022) mediram o desempenho do FreeRTOS em arquitetura multi-core no ESP32, focando em troca de contexto e jitter de tarefas, mas sem aprofundar a análise dos alocadores de heap disponíveis.

De forma semelhante, Tanović e Mezei (2024) avaliaram uma implementação paralela do algoritmo k-means no ESP32, comparando diferentes meios de armazenamento (Flash, PSRAM e cartão SD), mas com foco no desempenho do algoritmo de aprendizado de máquina, e não na estratégia de alocação de memória em si.

Essa lacuna é parcialmente abordada por O'Neill (2025), que desenvolveu um alocador multi-heap tolerante a falhas especificamente para o FreeRTOS, reconhecendo a importância de aprimorar os alocadores nativos do sistema, ainda que sem estabelecer uma comparação sistemática entre First-Fit e Best-Fit nas condições reais de uso do ESP32.

Esse conjunto de trabalhos reforça o obstáculo identificado na Introdução: a alocação de memória no ESP32/FreeRTOS raramente é o objeto central de investigação, permanecendo secundária a outros aspectos do sistema.

## 2.4 Avaliações experimentais comparativas entre First-Fit e Best-Fit

Por fim, um conjunto mais restrito de trabalhos se dedica especificamente à comparação experimental entre estratégias clássicas de alocação.

Ramakrishna et al. (2008) argumentam que, apesar de First-Fit e Best-Fit serem bem descritos teoricamente, faltam avaliações experimentais que considerem simultaneamente tempo de resposta e uso de memória sob cargas de trabalho representativas de aplicações reais, os autores propõem um alocador inteligente para sistemas embarcados que distingue objetos de vida curta e longa, validado por meio de uma carga de trabalho sintética (MIS) que buscou simular o comportamento de aplicações reais.

Mais recentemente, Septian et al. (2026) conduziram uma comparação híbrida entre First-Fit, Best-Fit e Worst-Fit no esquema de alocação MFT (Multiprogramming with a Fixed Number of Tasks), combinando testes estatísticos em datasets aleatorizados com análise visual no simulador Mishal23. Os resultados indicaram que o Best-Fit obteve a maior taxa de sucesso de alocação (84%), seguido por First-Fit (77%) e Worst-Fit (71%).

Embora relevante por fornecer uma comparação quantitativa direta entre os três algoritmos, o estudo foi conduzido em ambiente de simulação genérico, sem relação com hardware embarcado real ou com as particularidades de sistemas de tempo real, limitação reconhecida pelos próprios autores, que apontam a necessidade de validar os achados em sistemas mais complexos.

Esse cenário evidencia a lacuna central que motiva este trabalho: embora existam comparações teóricas clássicas Shore (1975), estudos com foco em previsibilidade Puaut (2002) e comparações quantitativas em ambientes simulados Septian et al. (2026); Ramakrishna et al. (2008), nenhum dos trabalhos revisados avalia First-Fit e Best-Fit diretamente em hardware embarcado real de baixo custo, executando o FreeRTOS sob condições controladas de carga. É exatamente essa lacuna que a metodologia descrita na próxima seção busca preencher.

## 3 Materiais e Métodos:

Para avaliar e comparar o desempenho temporal e o comportamento das estratégias de alocação de memória First-Fit e Best-Fit em ambiente de tempo real restrito, foi projetado e implementado um ambiente experimental controlado sobre o microcontrolador ESP32. A Figura 1 sintetiza a arquitetura metodológica do experimento, organizando as quatro etapas fundamentais do pipeline: a plataforma de hardware e ambiente de software, a integração dos alocadores customizados com heaps estáticos isolados, o gerador de carga de trabalho (*workload*) e o fluxo de monitoramento e análise estatística. Esta seção detalha cada um desses componentes.

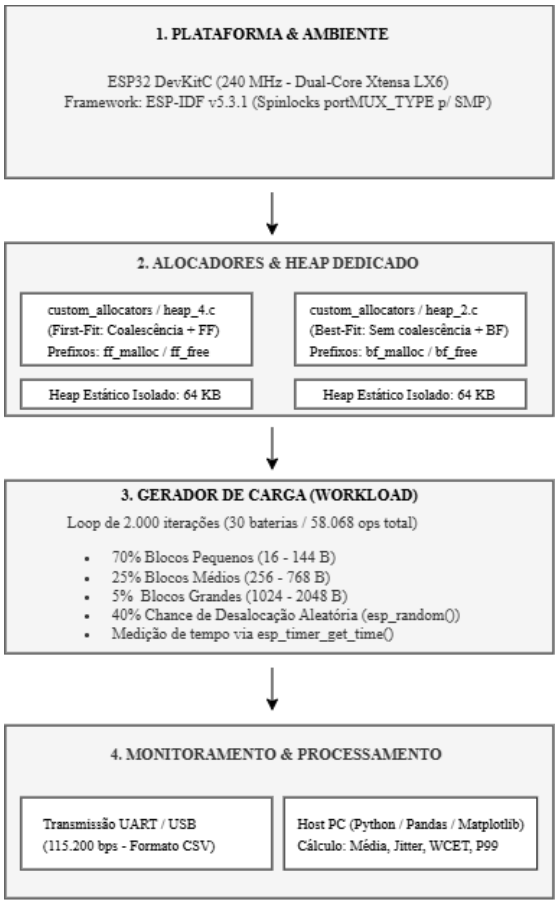



**Figura 1:** Fluxo metodológico do ambiente experimental e arquitetura de testes no ESP32.
Fonte: Os autores (2026).

### 3.1 Plataforma de Hardware e Ambiente de Desenvolvimento

O ambiente experimental utilizou duas placas de desenvolvimento ESP32 DevKitC (módulo ESP32-WROOM-32 com chip ESP32-D0WD-V3, revisão v3.1, arquitetura dual-core Xtensa LX6 operando a 240 MHz e 520 KB de SRAM interna). A utilização de duas unidades físicas do mesmo modelo teve como objetivo introduzir um controle de variação de fabricação de hardware, garantindo que os resultados obtidos não fossem fruto de idiossincrasias de um

único componente.

O desenvolvimento do firmware e o gerenciamento de compilação foram conduzidos via linha de comando utilizando o framework oficial da Espressif, o ESP-IDF v5.3.1, fixado em versão estável para assegurar a reprodutibilidade dos experimentos.

### 3.2 Integração Customizada dos Alocadores do FreeRTOS

Em suas versões recentes, o ESP-IDF substituiu os alocadores clássicos do FreeRTOS por um gerenciador unificado baseado em Two-Level Segregated Fit (TLSF). Para isolar e testar rigorosamente as estratégias First-Fit e Best-Fit originais, extraiu-se o código-fonte diretamente do repositório oficial do kernel do FreeRTOS (FreeRTOS-Kernel):

- `heap_2.c` (Best-Fit): Implementa a busca pelo menor bloco livre capaz de suprir a requisição, sem realizar a coalescência (fusão) de blocos livres adjacentes durante a desalocação.
- `heap_4.c` (First-Fit): Implementa a alocação no primeiro bloco livre com tamanho suficiente, realizando a coalescência de blocos contíguos na desalocação.

Estes arquivos foram encapsulados em um componente isolado denominado `custom_allocators`. As funções originais foram renomeadas com os prefixos `ff_` (First-Fit) e `bf_` (Best-Fit) para evitar conflitos com o alocador TLSF nativo do sistema operacional.

Cada alocador recebeu um heap próprio, estático e isolado de 64 KB (`configTOTAL_HEAP_SIZE = 65536`), garantindo que as operações de medição não interferissem na memória utilizada pelo núcleo do FreeRTOS e pelos drivers do ESP32. Além disso, as seções críticas foram adaptadas com spinlocks do tipo `portMUX_TYPE` para garantir suporte à execução multicore (SMP) do ESP-IDF.

### 3.3 Programa de Carga de Trabalho (Workload)

A carga de trabalho foi projetada para simular os padrões de requisição dinâmica de sistemas de IoT e tempo real, nos quais ocorrem frequentes alocações e liberações não ordenadas de memória. O experimento executa 2.000 iterações por bateria de teste, operando sob as seguintes diretrizes:

**Distribuição Ponderada de Tamanhos de Bloco:**

- **70% de blocos pequenos (16 a 144 bytes):** Simulando filas de mensagens, estruturas de tarefas e buffers de sensores.
- **25% de blocos médios (256 a 768 bytes):** Simulando pacotes de rede e buffers de comunicação serial.
- **5% de blocos grandes (1024 a 2048 bytes):** Simulando requisições esporádicas de alto volume (ex.: quadros de imagem ou buffers HTTP).

**Padrão Aleatório de Liberação:** A cada iteração, há 40% de probabilidade de disparar a desalocação de um bloco sorteado aleatoriamente entre os blocos ativos (via `esp_random()`), induzindo a formação de buracos (*gap formation*) e fragmentação externa real no heap.

**Aquecimento (Warm-up):** Uma alocação e desalocação inicial descartável é realizada antes do início da contagem das métricas para contornar a inicialização preguiçosa (*lazy initialization*) nativa do `heap_4.c`.

### 3.4 Métricas Coletadas e Protocolo de Execução

Para cada operação de alocação ou desalocação realizada, o firmware registra:

- **Tempo de Execução ($\mu$s):** Medido com resolução nominal de microssegundos via `esp_timer_get_time()`. Embora a resolução discreta do temporizador do hardware introduza ruído de quantização nas medições individuais, a agregação sobre o elevado volume amostral do experimento reduz o erro padrão da média a níveis desprezíveis, garantindo a validade estatística das diferenças observadas em escala sub-microssegundo.
- **Uso e Fragmentação de Heap:** Espaço livre restante após a operação.

Os dados foram transmitidos via interface serial (UART/USB) a 115.200 bps em formato CSV e capturados no computador host. Para garantir a significância estatística e controlar eventuais variações de fabricação do hardware, foram executadas 30 baterias de testes independentes distribuídas em duas unidades físicas do ESP32: 20 execuções na Placa 1 (unidade principal de amostragem) e 10 execuções na Placa 2 (unidade de validação cruzada de hardware). Essa distribuição assimétrica permitiu obter uma amostra densa do comportamento temporal no dispositivo principal, assegurando, simultaneamente, que os resultados fossem reprodutíveis e imunes a idiossincrasias de um único componente físico — hipótese confirmada pela convergência das distribuições observadas entre os dispositivos (Figura 2). Ao todo, foram registradas e higienizadas 58.068 operações. Esse total é inferior ao máximo teórico porque algumas requisições de alocação falham quando o heap não dispõe de um bloco compatível, especialmente no `heap_2`, que não realiza coalescência de blocos livres.

A análise dos dados foi processada via rotinas em Python (utilizando as bibliotecas `pandas` e `matplotlib`), extraindo métricas de Média, Desvio Padrão, Pior Caso (*Worst-Case Execution Time* – WCET) e Percentil 99 (P99).

## 4 Resultados e Discussão

A Tabela 2 apresenta a consolidação estatística obtida a partir das 58.068 linhas de dados processadas no experimento. A Figura 2 complementa essa síntese ao mostrar a distribuição dos tempos de alocação e desalocação.

### 4.1 Análise do Tempo de Alocação e Desalocação

Na operação de alocação (`malloc`), a estratégia Best-Fit obteve desempenho superior ao First-Fit em todas as métricas temporais: menor tempo médio (6,95 $\mu$s contra 7,35 $\mu$s), menor desvio padrão (1,81 $\mu$s contra 2,32 $\mu$s), pior caso mais contido (21 $\mu$s contra 24 $\mu$s) e P99 de 13 $\mu$s (frente a 15 $\mu$s). A menor variabilidade do Best-Fit no `malloc` indica maior previsibilidade temporal durante a criação de

**Tabela 2:** Estatísticas do tempo de execução das operações de memória no ESP32 ($\mu$s).

| Operação | Estratégia (Heap) | Média ($\mu$s) | Desvio Padrão ($\mu$s) | Pior Caso / WCET ($\mu$s) | Percentil 99 / P99 ($\mu$s) |
|---|---|---|---|---|---|
| Alocação (Malloc) | Best-Fit (`heap_2`) | 6,95 | 1,81 | 21,00 | 13,00 |
| | First-Fit (`heap_4`) | 7,35 | 2,32 | 24,00 | 15,00 |
| Desalocação (Free) | Best-Fit (`heap_2`) | 6,22 | 1,31 | 20,00 | 10,00 |
| | First-Fit (`heap_4`) | 5,63 | 0,72 | 16,00 | 8,00 |

Fonte: Os autores (2026).

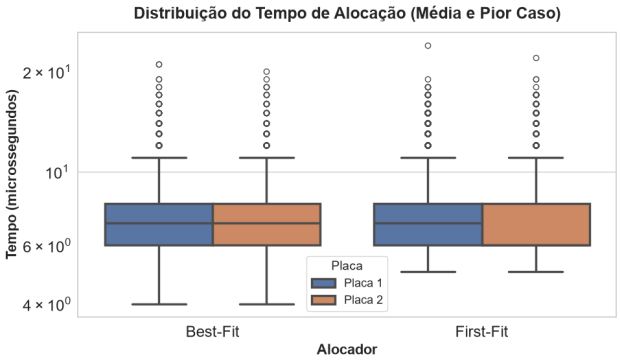



**Figura 2:** Distribuição e comparativo do tempo de execução de alocação (malloc) e desalocação (free) entre as estratégias First-Fit e Best-Fit no ESP32.

Fonte: Os autores (2026).

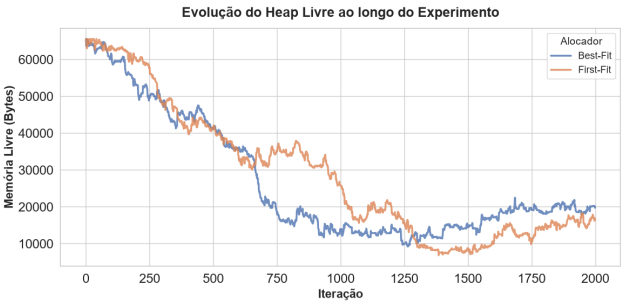



**Figura 3:** Evolução do consumo de heap livre e comportamento da fragmentação externa ao longo das iterações da carga de trabalho.

Fonte: Os autores (2026).

estruturas de dados.

Na operação de desalocação (`free`), entretanto, a dinâmica se inverte: o First-Fit superou o Best-Fit expressivamente. O First-Fit alcançou tempo médio de liberação de 5,63 $\mu$s (contra 6,22 $\mu$s do Best-Fit) e um desvio padrão de apenas 0,72 $\mu$s (frente a 1,31 $\mu$s do Best-Fit). O pior caso do First-Fit no `free` foi de 16 $\mu$s (contra 20 $\mu$s), e o P99 ficou em 8 $\mu$s (contra 10 $\mu$s).

## 4.2 Discussão dos Achados

Os resultados esclarecem aspectos fundamentais do comportamento dos alocadores clássicos em hardware embarcado sem MMU:

- **Mecanismo Estrutural do FreeRTOS e Desempenho no `malloc`:** Embora teoricamente o Best-Fit (`heap_2`) exija a inspeção de vários blocos para selecionar o menor espaço adequado (comportamento clássico $O(n)$), os dados observados revelaram um tempo de alocação ligeiramente inferior ao do First-Fit (`heap_4`) (6,95 $\mu$s vs 7,35 $\mu$s). Esse resultado contraintuitivo justifica-se por duas razões estruturais: (1) a predominância de blocos pequenos (70%) no *workload* pode ter reduzido o custo observado das varreduras da lista livre, mas não elimina a necessidade de percorrer a lista para que o `heap_2` selecione o menor bloco compatível; e (2) o `heap_4` precisa manter a lista estritamente ordenada por endereço para viabilizar a coalescência, o que exige um *overhead* adicional de reordenação de ponteiros e divisão (*splitting*) de blocos dentro de seções críticas protegidas por *spinlocks multicore*. Em contrapartida, na desalocação, a ordenação por endereço do `heap_4` viabiliza a fusão imediata de vizinhos, tornando seu `free` extremamente rápido e estável (0,72 $\mu$s de desvio padrão).
- **Previsibilidade em Tempo Real:** Conforme criticado por Puaut (2002), focar apenas na média oculta riscos de escalonamento. Em aplicações críticas operando no FreeRTOS, a baixíssima variabilidade do First-Fit no `free` (WCET = 16 $\mu$s, P99 = 8 $\mu$s) é uma propriedade valiosa para garantir que tarefas de encerramento de buffers cumpram seus *deadlines* sem estourar a janela de tempo.
- **Significância Estatística, Relevância Prática e Fragmentação:** A aplicação do teste *t* de Welch às 58.068 observações confirma que as diferenças observadas nos tempos médios de alocação (0,40 $\mu$s) e desalocação (0,59 $\mu$s) são estatisticamente significativas ($p < 0{,}001$), rejeitando a hipótese nula de igualdade entre as distribuições. Contudo, o cálculo do tamanho do efeito ($d$ de Cohen $< 0{,}2$) revela que, em termos operacionais em

hardware real, há uma equivalência prática no tempo médio de resposta entre Best-Fit e First-Fit, contrastando com discrepâncias mais acentuadas observadas em ambientes simulados genéricos Septian et al. (2026). O verdadeiro fator determinante reside na fragmentação: a ausência de coalescência no `heap_2` (Best-Fit), ilustrada pela evolução do heap livre na Figura 3, pode inviabilizar seu uso em aplicações *soak* de longa duração com alocações dinâmicas contínuas, sob risco de colapso do heap por fragmentação externa, enquanto o `heap_4` (First-Fit) absorve tais variações sustentando a execução do sistema.

## 5 Conclusões

Este trabalho apresentou uma avaliação experimental comparativa das estratégias clássicas de alocação dinâmica de memória First-Fit (`heap_4`) e Best-Fit (`heap_2`) em ambiente de tempo real, executadas sobre o microcontrolador ESP32 sob o sistema operacional FreeRTOS. A partir de um framework de testes controlado que processou 58.068 operações de memória, o estudo preencheu uma lacuna recorrente na literatura quanto ao comportamento dessas estratégias diretamente em hardware embarcado de baixo custo sem MMU.

As principais conclusões obtidas foram:

- **Desempenho de Alocação:** A estratégia Best-Fit apresentou melhor desempenho no `malloc`, com menor tempo médio (6,95 $\mu$s), menor *jitter* (desvio padrão de 1,81 $\mu$s) e pior caso mais contido (WCET = 21 $\mu$s), sendo mais indicada para aplicações que exigem previsibilidade estrita durante a fase de alocação de recursos.
- **Desempenho de Desalocação:** A estratégia First-Fit destacou-se no `free`, sendo mais rápida em média (5,63 $\mu$s) e significativamente mais estável (desvio padrão de 0,72 $\mu$s e WCET = 16 $\mu$s).
- **Recomendações Práticas para Engenharia de Software Embarcado:** Para aplicações de IoT no ESP32 com perfil dinâmico prolongado (como *gateways* ou nós sensores com reconexões frequentes), o First-Fit (`heap_4`) consolida-se como a escolha recomendada, pois a coalescência de blocos evita o colapso do heap por fragmentação, mantendo tempos de liberação previsíveis. Para sistemas com padrão de alocação inicial fixo e poucas liberações posteriores, o Best-Fit (`heap_2`) oferece ligeira vantagem de tempo na fase de inicialização.

Como trabalhos futuros, pretende-se expandir a avaliação experimental para comparar o desempenho dos alocadores nativos do FreeRTOS com a implementação baseada em TLSF (padrão do ESP-IDF) e com alocadores modernos como o RT-Mimalloc, avaliando o impacto do consumo de memória RAM adicional exigido pelos cabeçalhos dessas estruturas avançadas.

## Agradecimentos

Gostaria de expressar meus sinceros agradecimentos ao Instituto Federal de Santa Catarina (IFSC) pela oportunidade ímpar de aprendizado e crescimento acadêmico. Todo o desenvolvimento deste trabalho, bem como a base de conhecimento necessária para a sua realização, só foram possíveis graças ao ensino de excelência, à estrutura e ao apoio contínuo proporcionados pela instituição. Este projeto representa não apenas um esforço de pesquisa, mas o reflexo da formação técnica e científica construída ao longo da minha trajetória no IFSC.

## Referências

Arm, J. et al. (2022). Measuring the performance of freertos on esp32 multi-core, *IFAC-PapersOnLine* **55**(4): 293–298.
https://doi.org/10.1016/j.ifacol.2022.06.048

Bukkapatnam, K. et al. (2020). Smart memory management (samm) for embedded systems without mmu, *IOP Conference Series: Materials Science and Engineering*, Vol. 981, p. 032010.
https://doi.org/10.1088/1757-899X/981/3/032010

Crespo, A., Ripoll, I. e Masmano, M. (2006). Dynamic memory management for embedded real-time systems, *in* B. Kleinjohann et al. (eds), *From Model-Driven Design to Resource Management for Distributed Embedded Systems*, Springer, Boston, pp. 19–32.
https://doi.org/10.1007/978-0-387-39362-9_21

Deligiannis, I. e Kornaros, G. (2016). Adaptive memory management scheme for mmu-less embedded systems, *Proceedings of the 11th IEEE Symposium on Industrial Embedded Systems (SIES)*, IEEE, Krakow, pp. 1–8.
https://doi.org/10.1109/SIES.2016.7509439

Giannessi, R., Biondi, A. e Biasci, A. (2024). Rt-mimalloc: A new look at dynamic memory allocation for real-time systems, *Proceedings of the 30th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, IEEE, Hong Kong, pp. 173–185.
https://doi.org/10.1109/RTAS61025.2024.00022

Goyette, R. (2007). An analysis and description of the inner workings of the freertos kernel, *Technical report*, Carleton University, Department of Systems and Computer Engineering, Ottawa.

Musaddiq, A. et al. (2018). A survey on resource management in iot operating systems, *IEEE Access* **6**: 8459–8482.
https://doi.org/10.1109/ACCESS.2018.2808324

O'Neill, G. E. (2025). *A fault-tolerant, multi-heap dynamic memory allocator for freertos*, Master's thesis, California Polytechnic State University, San Luis Obispo.
https://digitalcommons.calpoly.edu/theses/3099/

Puaut, I. (2002). Real-time performance of dynamic memory allocation algorithms, *Proceedings of the 14th Euromicro Conference on Real-Time Systems (RTS)*, IEEE, Vienna, pp. 41–49.
https://doi.org/10.1109/EMRTS.2002.1019184

Ramakrishna, M. et al. (2008). Smart dynamic memory allocator for embedded systems, *Proceedings of the 23rd*

*International Symposium on Computer and Information Sciences (ISCIS)*, IEEE, Istanbul, pp. 1–6.
https://doi.org/10.1109/ISCIS.2008.4717922

Rebel, G. et al. (2013). Fast constant time memory allocator for inter task communication in ultra low energy embedded systems, *Proceedings of the 9th Conference on Ph.D. Research in Microelectronics and Electronics (PRIME)*, IEEE, Villach, pp. 149–152.
https://doi.org/10.1109/PRIME.2013.6603114

Septian, F. A. et al. (2026). Efficiency analysis of first fit, best fit, and worst fit algorithms in mft memory allocation using the mishal23 simulator, *TESLA: Jurnal Teknik Elektro* **28**(1): 31–39.
https://doi.org/10.24912/tesla.v28i1.37738

Shore, J. E. (1975). On the external storage fragmentation produced by first-fit and best-fit allocation strategies, *Communications of the ACM* **18**(8): 433–440.
https://doi.org/10.1145/360933.360949

Tanović, A. e Mezei, I. (2024). Embedded parallel k-means algorithm evaluation on esp32 across various memory allocations, *Proceedings of the IEEE East-West Design & Test Symposium (EWDTS)*, IEEE, Yerevan, pp. 1–7.
https://doi.org/10.1109/EWDTS63723.2024.10873639